

DIVIDE AND CONQUER: MERGESORT

Link to practice handout

<https://bit.ly/Divide-and-Conquer-Practice>

Divide and Conquer Algorithms

Algorithm Approach:

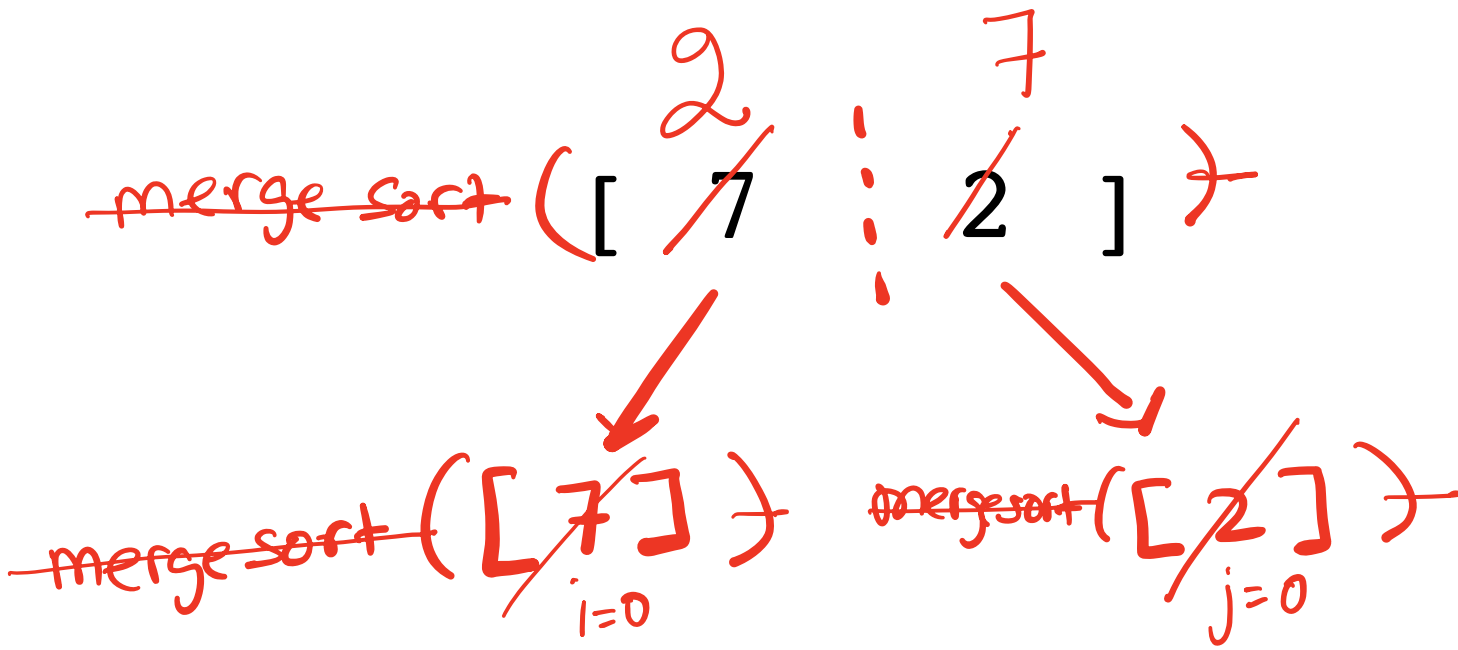
- **Divide** a large problem into sub-problems
- **Solve** each sub-problem
- **Combine** the solutions of sub-problems to obtain the solution for the original problem

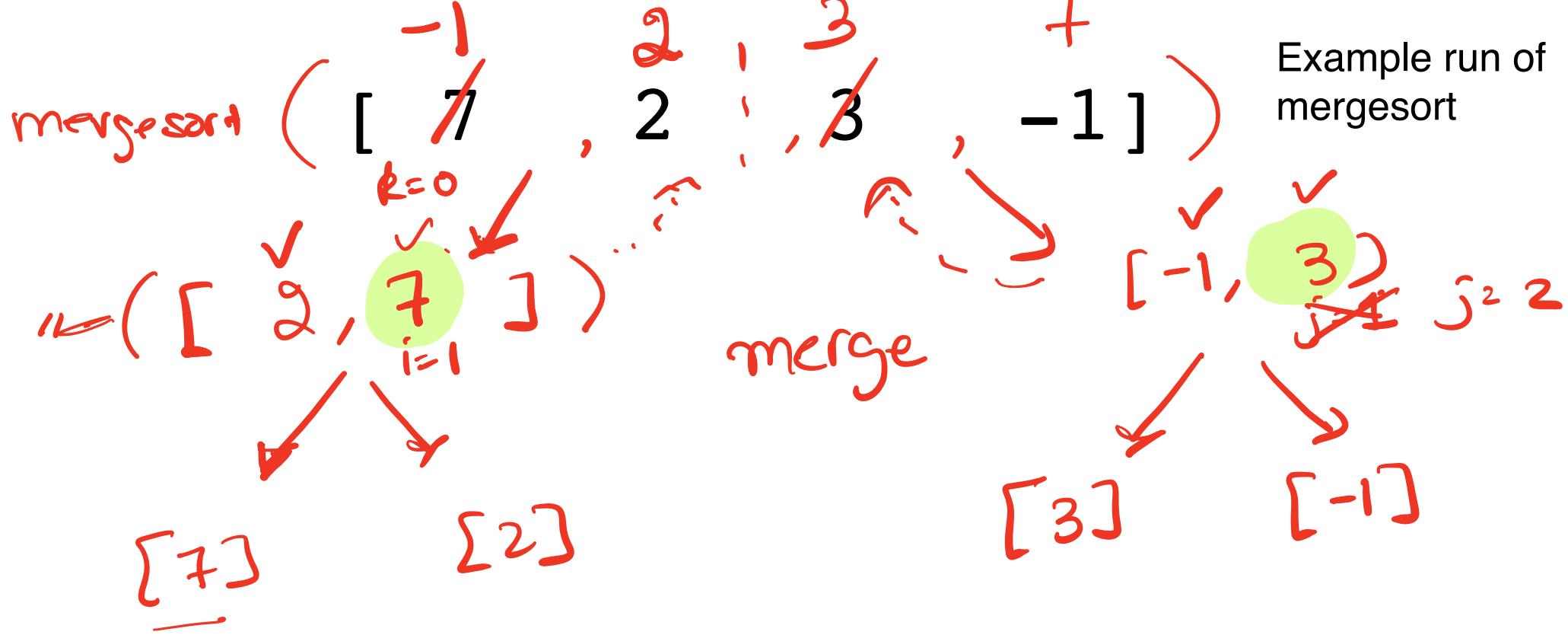
Merge Sort Algorithm

`MergeSort(vector v)`

- Divide `v` into left half and right half
- Sort the left half, then sort the right half
- Combine (merge) the two sorted halves

Example run of mergesort





level 0

[7 2 5 3 -1]

level 1

[7 2]

[5 3 -1]

[5 3]

[-1]

level 2 [7 7]

[2]

[5 3]

[-1]

[5] [3]

$\log(n)$

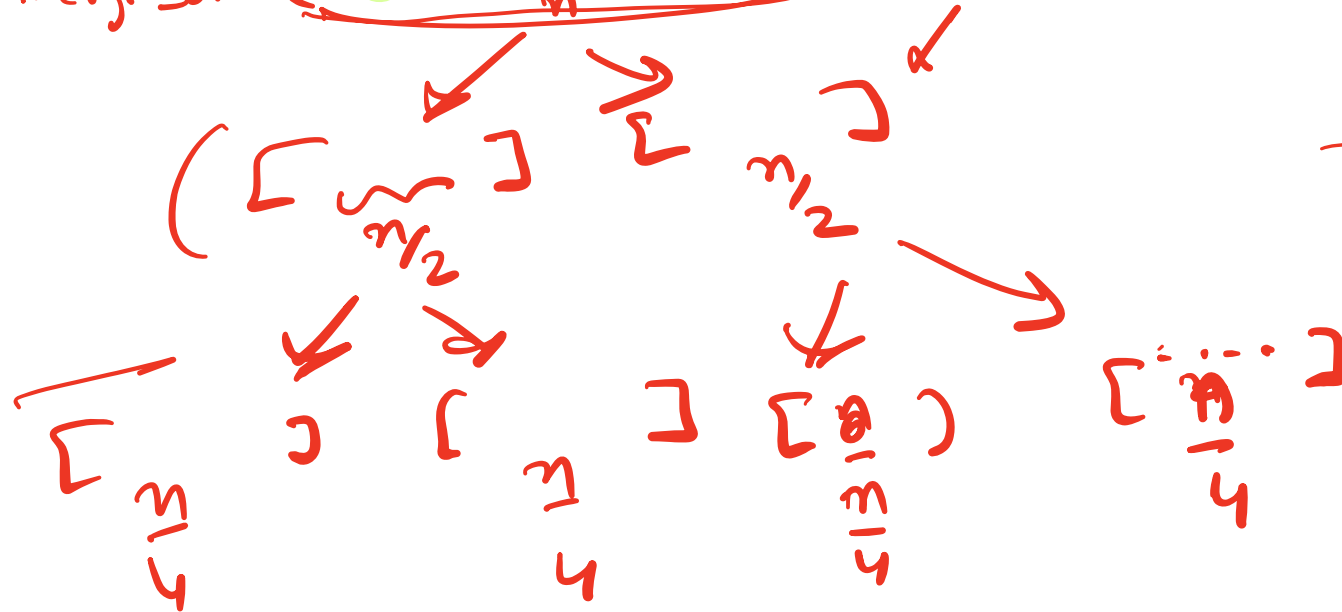
Activity 1: Trace merge sort on the above example input with 5 elements
Draw the final binary tree representation of the trace.
What is the height of the resulting binary tree?

Time Complexity of Mergesort

$T(n)$: Time it takes to compute mergesort ($[\dots]$ n)

Approach: Total work done over the entire run of the algo

merge sort ($[\dots]$ n)



Total

Cost to split
(per level)

n

Cost to merge
(per level)

n

n

n

n

n

$n \log n$

$n \log n$

Total work done is (1) splitting (2) merge

split cost

Split cost
How much copying happens across the entire algorithm?
array size $n \rightarrow n$ copies

Top level: split 1 vector of size $n \rightarrow n$ copies
 split 2 vectors of size $\frac{n}{2} \rightarrow 2 + \frac{n}{2} = n$

Top level: split 4 vectors of size $\frac{n}{2} \rightarrow 2 \times \frac{n}{2} = n$
 next level: split 2 vectors of size $\frac{n}{2} \rightarrow 4 \times \frac{n}{2} = n$

next level: $\frac{n}{4} \rightarrow 4 \times \frac{n}{4} = n$

How many levels? $\log(n)$

How many levels
Total cost to split = $O(n \cdot \log n)$

Merge cost

Merge cost
top level merge ([2/3] , [2/3]) $\rightarrow$ n placements
merge $\rightarrow$

next level merge 2 pairs of $\left(\frac{n}{4}, \frac{n}{4}\right) \rightarrow 2 \times \frac{n}{2} = n$

4 merge 4 pairs of $(\frac{n}{2}, \frac{n}{2}) \rightarrow 4 \cdot \frac{n}{4} = n$

1

 $\log(n)$

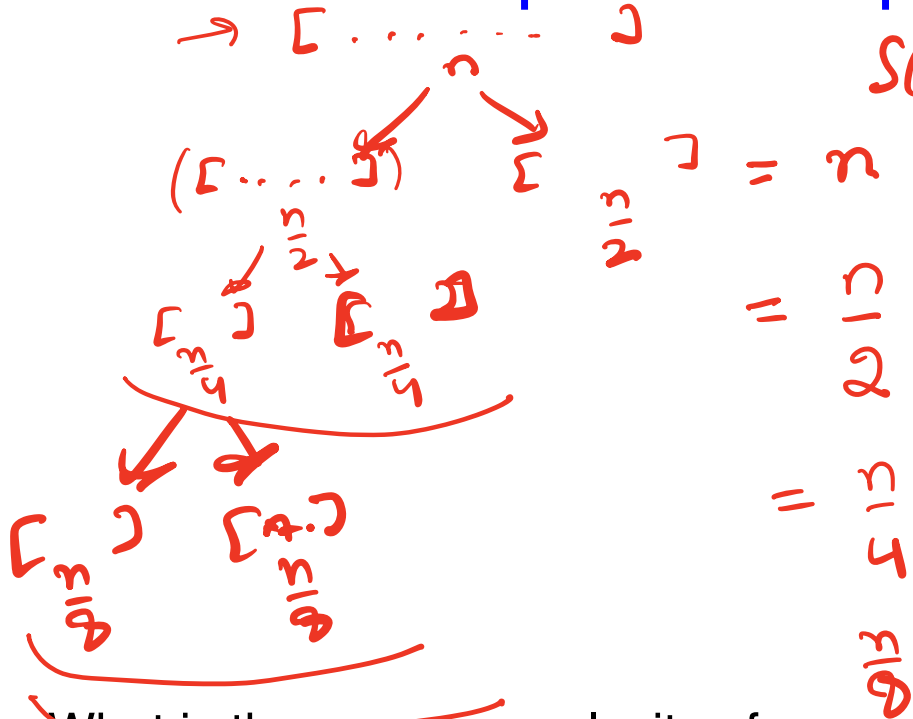
$\therefore$
 $\log(n)$
 Total cost to merge $= O(\log^n \cdot n)$
 $T(n) = O(n \log n) + O(n \log n)$
 $= O(n \log n)$

to merge $\approx O(\log^n \cdot n)$

$T(n) = O(\overset{\text{cost to split}}{n \log n}) + O(\overset{\text{cost to merge}}{n \log n})$

$\quad \quad \quad = O(n \log n)$

Space Complexity of Mergesort



$S(n)$: Extra space used (besides the input) when the recursion reaches max depth

$$S(n) = \left(n + \frac{n}{2} + \frac{n}{4} + \frac{n}{8} + \dots \right)$$

$$= n \left(1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots \right)$$

converge to a const

$$= O(n)$$

What is the space complexity of mergesort?

- A. $O(n)$ ←
- B. $O(n \log n)$
- C. $O(2^n)$
- D. Something else

Heap Sort



Source: <https://cosmictechie.hashnode.dev/heap-sort>

HeapSort Algorithm

HeapSort(vector v)

```
for i from n/2-1 down to 0: ] // heapify: build max heap  
    bubble_down(v, n, i)
```

$O(n)$: analyzed in a previous lecture

```
for i from n-1 down to 1: ] // Invariant:  
    swap(v[0], v[i])  
    bubble_down(v, i, 0)
```

$O(\log n)$ $\rightarrow O(n \log n)$ heap: $v[0..i-1]$
sorted: $v[i..n-1]$

Heapsort: <https://visualgo.net/en/heap>

Total runtime = $O(n \log n)$
Space complexity = $O(1)$

Why Mergesort over Heapsort?

- Same $O(n \log n)$ time, but heapsort has poor cache performance - why?
- Mergesort is **stable**: equal elements keep their original relative order
- Mergesort parallelizes naturally